In-Hand Manipulation of a Cube with Allegro Dex-4 Hand

EECE 5552: Project Report

Sasha Oswald
oswald.s@northeastern.edu

Chengpeng Jin
jin.chengp@northeastern.edu

Kyle Lou
lou.ky@northeastern.edu

Jorge Ortega
ortega.jor@northeastern.edu

Arjun Viswanathan
viswanathan.ar@northeastern.edu

December 14, 2025

1 Introduction

Manipulation is a very valuable and necessary task that humans are capable of performing. It allows us to go about our daily lives without a second thought of just how complex it is. The ability to hold and reorient objects using just one's fingers, or picking up and placing objects. But there are many individuals who are unable to perform manipulation with their fingers, be it from a disability or an accident leading to injury or amputation.

Our work here presents the control and reorientation of a cube with a 4-fingered robotic manipulator. It can act as a substitute for these humans and help them perform daily tasks involving inhand manipulation. Users can prefer to use robotic manipulators as prosthetics or as commandable objects that help them without being attached to their body.

We present two different paradigms for the in-hand manipulation task. The first is derived fully from classical controls, featuring a combination of impedance control, Quadratic Programming (QP) stabilization, and nullspace control. The second will be fully Reinforcement Learning (RL). Through the design of rewards, penalties, and domain randomization, we show how it is possible to achieve the same dexterity on the robotic manipulator, but more robust, being capable of handling model mismatches, friction and mass differences, and even external disturbances. We use MuJoCo for our force control simulation, and IsaacLab for our RL policy discovery.

2 Robot Platform

We use the Allegro 4-fingered dexterous hand for our simulations [1]. The object of choice is a cube with different patterns on its faces, which allows us to determine if the cube is facing the right orientation as commanded easily. Figure 1 shows the hand holding the cube.

3 Methods

3.1 Force Control

The force controller is implemented from previous work published in IEEE Robotics and Automation Letters [2]. The paper intorduces a novel object-level control framework for the dexterous in-hand manipulation of objects with torque controlled robotic hands. It uses a combination of impedance control, QP force control, and nullspace control to balance the object and reorient it to a desired orientation.

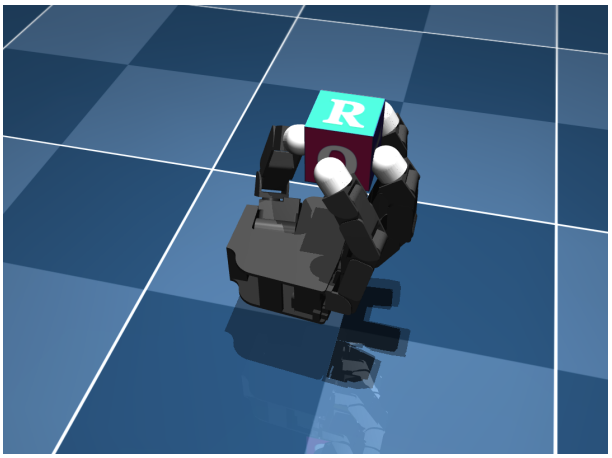


Figure 1: Allegro 4-Fingered Manipulator

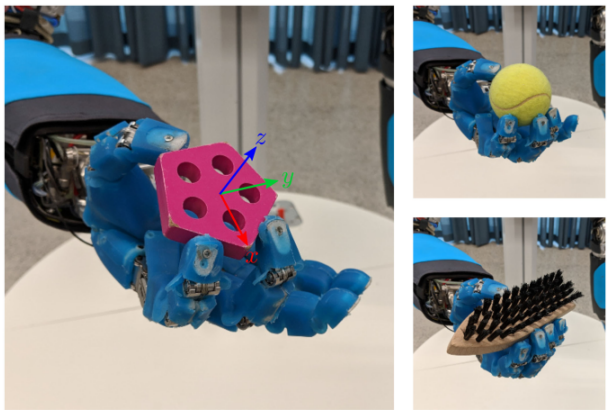


Figure 2: DLR 4-fingered robot handling different objects using the force controller from the paper

Figure 3 shows how the force control on the object maps to joint torques on each link of the manipulator. If the force on the fingertips (end-effectors) are known, then it can be mapped to torques on the joints using the Jacobian

$$\tau = J^T f \tag{1}$$

The pose of the object is expressed by vector $x \in \mathbb{R}^6$, representing the position and orientation w.r.t the inertial frame I. The vector of all contact positions is denoted by $c \in \mathbb{R}^{3n}$. The vector of joint positions is represented by $q \in \mathbb{R}^m$ for m joints.

The changes in object pose are denoted by a twist vector $v \in \mathbb{R}^6$. This can be related to the vector of contact velocities $\dot{c} \in \mathbb{R}^{3n}$ using the grasp matrix $G \in \mathbb{R}^{6\times 3n}$

$$\dot{c} = G^T v \tag{2}$$

The hand Jacobian $J \in \mathbb{R}^{3n\times m}$ relates the joint

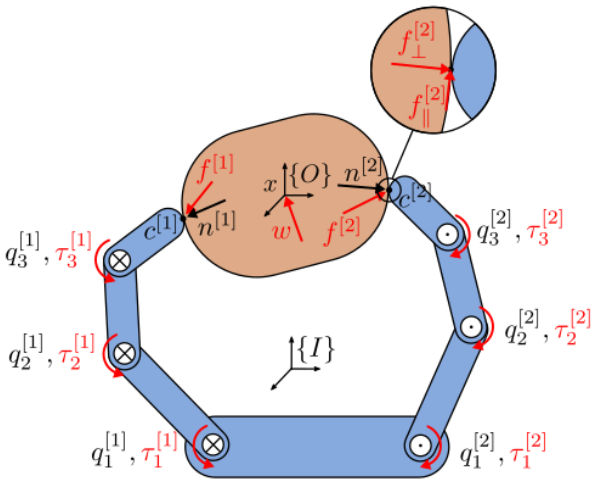


Figure 3: Main quantities of a fingertip grasp of an object

velocities $\dot{q}$ to contact velocities

$$\dot{c} = J\dot{q} \tag{3}$$

and also the relationship between the forces and torques. The object wrench that is generated by a set of forces $f \in \mathbb{R}^{3n}$ can be calculated using G

$$w = G^T f \tag{4}$$

So then, we can express the dynamics of the grasp system as follows:

$$M_o(x)\ddot{x} + C_o(x,\dot{x})\dot{x} + g_o(x) - G^T f = w_{ext} \tag{5}$$

$$M_f(q)\ddot{q} + C_f(q,\dot{q})\dot{q} + g_f(q) + J^T f = \tau_{ext} + \tau_{des} \tag{6}$$

Where $M_o \in \mathbb{R}^{6\times6}$, $M_f \in \mathbb{R}^{m\times m}$ are the inertia matrices, $C_o \in \mathbb{R}^{6\times6}$ and $C_f \in \mathbb{R}^{m\times m}$ are the Coriolis matrices and $g_o, g_f \in \mathbb{R}^6, \mathbb{R}^m$ are vectors of gravitational loads on the object and finger links. External forces applied are described by the vectors $w_{ext} \in \mathbb{R}^6$ and $\tau_{ext} \in \mathbb{R}^m$. Then, the actuator torques are given in $\tau_{des} \in \mathbb{R}^m$.

Both the desired object pose and normal contact force for each contact is provided by the user. Then, the specific force vector for each contact is dynamically calculated by the controller to balance the forces on the object and consider friction constraints. So given the friction constraints

$$f_{||}^{[i]} \leq \mu f_{\perp}^{[i]} \tag{7}$$

$$f_{max} \geq f_{\perp}^{[i]} \geq f_{min} \tag{8}$$

Existing contacts are actively released in order to reposition a finger during finger gaiting.

3.1.1 Impedance Controller

The purpose of this controller is to position the fingers in a way that results in object motion torwards desired pose, while maintaining the initial grasp to avoid slippage of the fingers on the object. Small changes in pose of the object result in changes in contact positions. This can approximated with

$$\Delta c_x = G^T W^{-1} \Delta x \tag{9}$$

where $W \in \mathbb{R}^{6\times6}$ is the matrix that maps the object twist to changes in its position and Euler angles. So then the desired change in contact points can be reformulated as

$$\Delta c_x = G^T W^{-1}(x_d - x) \tag{10}$$

But the contact configuration may gradually change over time because of any slippage. Therefore, there needs to be an additional contact position change Δc_f to correct the finger configuration w.r.t the initial configuration without affecting the object motion

$$\Delta c_f = (I - G^+ G)(c_o - c) \tag{11}$$

where $I \in \mathbb{R}^{3n\times3n}$ is the identity matrix, $(G^T)^+$ is the Moore-Penrose inverse of G^T, and c is the vector of current contact positions. c_0 is the contact positions in the initial grasp configuration in the inertial frame. It is time-constant. So moving the contacts by Δc_f does not change the pose of the object, so it is a projection in the nullspace of the grasp matrix.

The combined desired change in contact positions is given as

$$\Delta c = \Delta c_x + \Delta c_f \tag{12}$$

Then the impedance behavior is realised by formulating a spring damped system that yields the desired contact forces f_x

$$f_x = K_x \Delta c + D_x \Delta \dot{c} \tag{13}$$

where K_x, D_x are chosen by the user.

3.1.2 QP Force Stabilizer

The impedance controller managed posing the object to the desired pose, but does not handle internal forces or compensate additional loads on the object. The QP controller here will calculate contact forces needed to balance the dynamical loads on the object while obeying friction cone constraints.

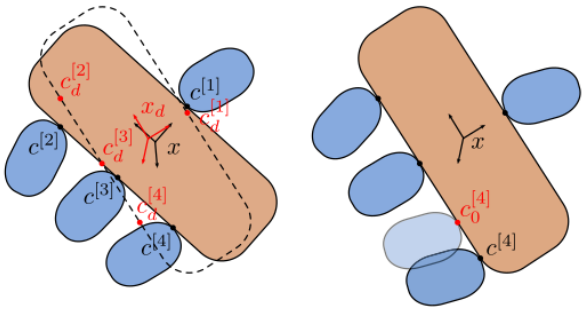


Figure 4: Using impedance control to reorient the pose, while switching contact of finger to keep object from slipping. The red desired contact is where the put the finger to keep it from slipping

So if $w_{dyn} \in \mathbb{R}^6$ is the sum of dynamic object loads, given by

$$w_{dyn} = M_o(x)\ddot{x} + C_o(x, \dot{x})\dot{x} + g_o \tag{14}$$

The vector of desired normal forces is denoted $f_d \in \mathbb{R}^{3n}$ and calculated for each contact, using its normal direction. So the vector can be expressed by the sum of two distinct terms

$$f = G^+ w + (I - G^+ G) f_* \tag{15}$$

where the first term describes a set of minimal forces to generate a desired wrench w on the object and the second term projects a force f_* into the nullspace of the grasp matrix. So then, the problem of determining the internal forces f_{int} is formulated as finding f_* as

$$\min_{f_*} ||f_* - f_d|| \tag{16}$$

such that

$$f_{int} = G^+ w + (I - G^+ G) f_* \tag{17}$$

$$f_{int} = f_{dyn} + (I - G^+ G) f_* \tag{18}$$

subject to friction cone constraints the same way as with the impedance controller. The minimization here is now equivalent to the QP problem of the form

$$\min_{f_*} (\frac{1}{2} f_*^T H f_* - f_*^T f_d) \tag{19}$$

subject to a set of linear equality constraints

$$f_{*,||}^{[i]} - \mu f_{*,\perp}^{[i]} \leq -(f_{dyn,||}^{[i]} - \mu f_{dyn,\perp}^{[i]}) \tag{20}$$

$$f_{max} - f_{dyn,\perp}^{[i]} \geq f_{*,\perp}^{[i]} \geq f_{min} - f_{dyn,\perp}^{[i]} \tag{21}$$

Once we have the solutions from the impedance control and the QP, we add them together to get the computed force

$$f_c = f_x + f_{int} \tag{22}$$

and then extract the torques using the Jacobian

$$\tau_c = J^T f_c \tag{23}$$

3.1.3 Nullspace Controller

In the case of three of more DoF, there exists a nullspace in the force mapping. This means that the joints of the fingers would be unconstrained in this space and can drift into joint limits, so the manipulability of the object is reduced. So one can add a torque $\tau_{null} \in \mathbb{R}^m$ to the nullspace to cancel this effect.

$$\tau_{null} = K_{null} \Delta q^{[j]} + D_{null} \Delta \dot{q}^{[j]} \tag{24}$$

with

$$\Delta q^{[j]} = \frac{1}{2} \begin{bmatrix} 0 \\ 0 \\ q_4^{[j]} - q_3^{[j]} \\ q_3^{[j]} - q_4^{[j]} \end{bmatrix} \tag{25}$$

where $q_3^{[j]}$ and $q_4^{[j]}$ are the third and fourth joint angle, respectively. So this controller regulates the two joints to maintain the same value.

Now with τ_{null}, it can be combined with the original torques to obtain the final torque output.

$$\tau_{des} = \tau_c + (I - J^T (J^T)^+) \tau_{null} \tag{26}$$

This adds torques to the nullspace of the Jacobian and does not affect the contact forces on the object.

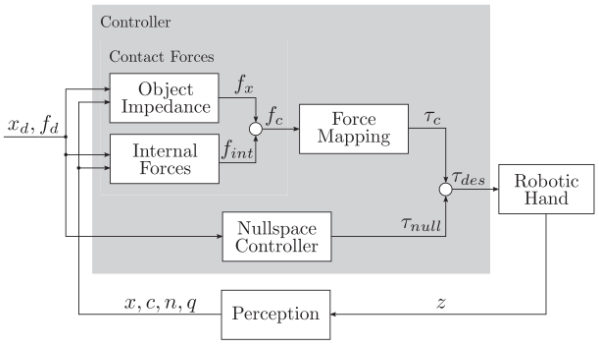


Figure 5: Overall control framework for force control

3.2 RL Control

The objective of the RL problem is to keep re-orienting the cube such that the goal orientations commanded are matched as quickly as possible. It will be initialized from different starting points, different frictions, masses, and feedback control gains. This allows the learned policy to have really good sim2real performance, as it has learnt to work in a variety of different conditions. We use Proximal Policy Optimization (PPO) [3] for our learning process.

3.2.1 PPO

PPO is an on-policy method used for training stochastic policies [3]. In particular, we are using the PPO-Penalty variant. It address the issue of performance collapse due to Q value overestimation in other methods such as DDPG. This algorithm learns both the policy parameters θ and advantages A. The policy π_θ (actor) is optimized using gradient ascent with the following objective function:

$$L_{actor}(\theta) = \mathbb{E}\left[\frac{P(a|\pi_\theta(s))}{P(a|\pi_{\theta_{old}}(s))}A - \beta \cdot \mathtt{KL}\right] \tag{27}$$

Where the Kulback-Leiber (KL) divergence of the two stochastic policy distributions is defined as:

$$\mathtt{KL} = -\int P(a|\pi_{\theta_{old}}(s)) \ln \frac{P(a|\pi_\theta(s))}{P(a|\pi_{\theta_{old}}(s))} da \tag{28}$$

The critic function with parameters ϕ is optimized by using gradient descent on the mean squared TD error (MSE):

$$L_{value}(\phi) = \frac{1}{2}\Big(V_\phi(s_t) - G\Big)^2 \tag{29}$$

where G is the discounted return for the entire episode (estimated either using Monte Carlo or GAE).

Therefore the total loss is:

$$L(\theta, \phi) = -L_{actor}(\theta) + L_{value}(\phi) \tag{30}$$

The PPO hyperparameters for our policy are shown in table 1.

3.2.2 Commands

The command input is a quaternion goal orientation. It will be randomly sampled from the euler angle counterparts, thus ensuring the quaternions are well-conditioned. Upon successful orientation of the cube by the policy, the next orientation is sampled. When the episode ends, the next orientation is sampled as well.

Since we are interested only in the re-orientation of the object in this project, meeting a desired position along with the orientation is not explored, although it can be easily added on with a robot performing a task, like pick and place.

3.2.3 Actions

The action output is an Exponential Moving Average (EMA) joint position command. The raw values out of the policy network are clamped within $[-1.0, 1.0]$

$$a_t = [a_1, a_2, ..., a_N] \in [-1.0, 1.0] \tag{31}$$

Then the EMA is applied to these, clamping the output to joint limits

$$\mathbf{q_{des}} = \alpha \mathbf{a_t} + (\mathbf{1} - \alpha)\mathbf{a_{t-1}} \in [\mathbf{q_{min}}, \mathbf{q_{max}}] \tag{32}$$

This ensures the actions are smooth and within the joint limits, so the policy cannot command a large value outside the limits which will map to a large torque on the joints and break the object being manipulated.

3.2.4 Observations

The observation vector allows the policy to see the environment state, and plan its actions accordingly. If there is a reward for something, it must be observed for best performance. We observe the full kinematic state of the object relative to the hand, as well as the hand joint positions and velocities, allowing us to plan correct joint position actions. The vector is shown in table 2.

3.2.5 Rewards and Penalties

Since we are only tracking the orientation command, a reward that minimizes the error is sufficient to learn the behavior. To help the policy succeed and have a greater incentive to meet the target orientation, we provide a bonus reward once it reaches close enough to the goal orientation.

At the same time, we add joint level regularization penalties to prevent excessive jitter while controlling the cube, and moving the joints too fast to perform the task. The rewards and penalties are shown in table 3.

3.2.6 Terminations

The terminations in table 4 are conditions that end the episode. These can be good, like timing out by

reaching the end of the episode without failing, or achieving N consecutive successes, or bad, like if the object is not within the operating space of the fixed hand. The primary idea is the end the episode, meaning the maximum reward that can be achieved is reduced, if the bad termination fires, so the policy learns to avoid that behavior by all means.

3.2.7 Domain Randomization

To bridge the sim2real gap and provide generality, we add randomizing terms to our policy setup, shown in table 5. These terms cover any model mismatch that can exist as well as handling a variety of initial starting points.

We also randomize the mass of the links and the object, so the policy learns to work with mass differences too.

4 Results and Discussion

4.1 Force Control

Figures 8, 9, 10 show the forces plots given by our impedance control, QP force control, and object pose tracking to hold the object in place. Figure 6 shows snapshots of the allegro hand performing the hold, along with normal force vector arrows shown in MuJoCo [4].

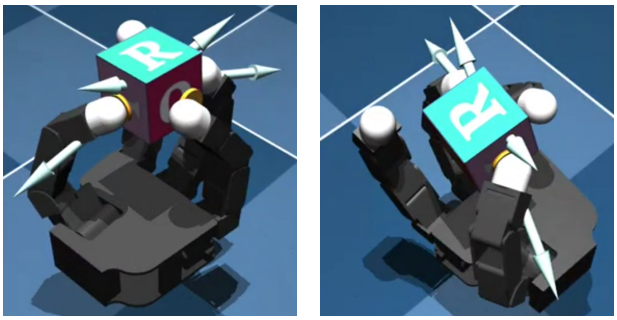


Figure 6: Snapshots from MuJoCo for force control results.

We were able to get the cube to be held in a stable manner with the allegro hand, with a little bit of drift from the original pose over a long time. For our force control piece, we noticed that the QP pushes the fingers on the side of the cube away, while using only the middle and thumb fingers to hold the cube. The forces are sane, and the cube is held for a long time without failing. But we were not able to fully debug why the QP chooses to push the fingers on the sides of the cube (index and ring fingers) away. The contact normals are all pointed to the outside of the cube, so our QP solution is a reaction force. So to obtain the forces the fingers apply on the cube, we simply negate f_*.

Additionally, our nullspace torques blow up the system really quickly, injecting massive torques in addition to the impedance and QP forces mapped into torques. We have followed the equations from the paper to write our code. But given that we had to make some changes to the equations to get the matrix math to agree, we speculate that the given equations are missing some components, or the dimensionality of one or more quantities are wrong.

Finally, the paper we are using for this approach does not outline how their grasp reconfiguration process works. The authors claim that their reconfiguration logic is provided in another work, but do not cite it or give a brief outline on how it should be implemented. The lack of an open-source codebase for this literature makes the implementation that much more harder.

4.2 RL Control

Figure 7 shows our controller result snapshots in IsaacLab [5] in tracking the object pose. Figure 11 shows the plot of tracking the object pose.

We were able to discover a robust RL policy that could use the full hand and manipulate the pose of the cube consistently. Our plots show that the object pose gets to the goal pose within 1-2 seconds, and continues to reach as many randomly sampled poses as possible.

Very rarely do we observe the policy is unable to hold onto the object and shoots it out of the hand in simulation. This is because we did not add a reset library to reset the robot joint positions and the cube root position. Randomly assigning the joints and cube pose will lead to poor learning, as there can be many cases where the cube spawns outside of the reachability of the hand and the episode terminates immediately. Unlike a locomotion environment training, we have to be smarter about how we reset the system, so a reset library will consist of specific starting positions that will always keep the start point of the system meaningful while allowing enough randomization for sim2real bridging.

We also note that the fingers jitter quite a bit while executing the joint position commands. This can be chalked up to the smoothness being too low. Since we are using the EMA joint position action, we can tune the value of α such that the actions applied into the robot are smoother.

Finally, we can also train a policy trivially by enforcing only the fingertips hold the object. This way,

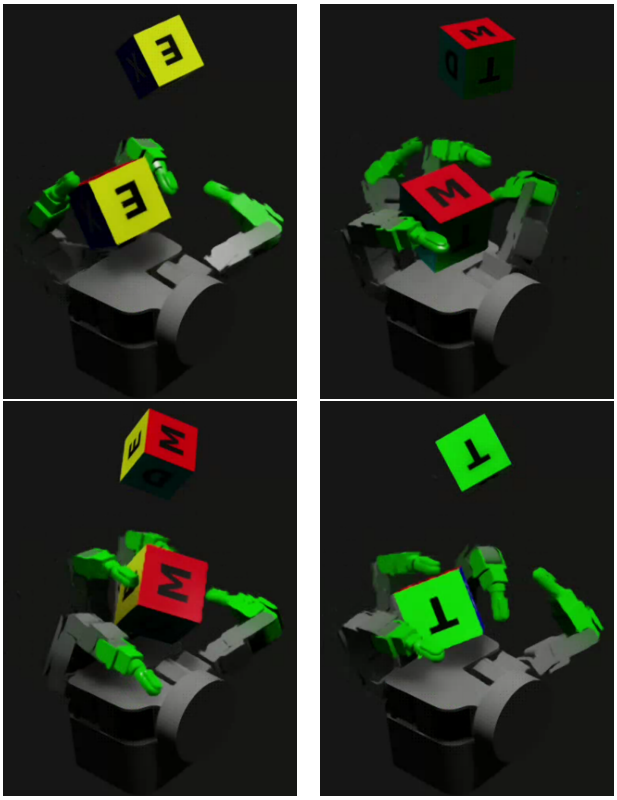


Figure 7: Snapshots from IsaacLab for RL control results. The goal poses are shown above the hand. Snapshots were taken when the policy got close to the goal.

we can obtain a result similar to the classical approach. But for the purposes of our project, we deem the policy using its full hand to manipulate the cube as satisfactory to solve the task. In most human tasks that involve inhand manipulation, use of the full hand is common. Using the fingertips alone increases the risk of dropping objects.

5 Conclusion and Future Work

In this project, we presented two paradigms for inhand manipulation of an object using the Allegro 4-fingered dexterous manipulator. Our work showed the dynamics and mathematical formulation of the control framework to move the object's pose, manage internal forces to hold the object's pose, and avoid singularities in the Jacobian by inserting torques into the nullspace. We also showed a fully RL formulation and how it is more robust to handling objects with unknown mass and inertia under a variety of initial states and model mismatches to achieve the same task.

As part of our future work, we plan to attach this to an arm on a robot, and perform pick and place tasks while manipulating the pose of the object being placed. This will allow us to perform full upper body control on a human task and automate its execution. One other addition from that point is to add a lower body controller to be able to move the legs, so a humanoid can now move around a work area and pick and place objects while manipulating the poses of the objects.

We provide all our code for force and RL control open-source on GitHub [6]. Additionally, the final project presentation slide deck is also provided [7].

References

[1] Wonik Robotics, "Allegro hand," https://www.wonikrobotics.com/research-robot-hand, 2014, accessed: 2025-01-XX.

[2] M. Pfanne, M. Chalon, F. Stulp, H. Ritter, and A. Albu-Schäffer, "Object-level impedance control for dexterous in-hand manipulation," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 2987–2994, 2020.

[3] O. SpinningUp, "Proximal Policy Optimization." [Online]. Available: https://spinningup.openai.com/en/latest/algorithms/ppo.html

[4] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 5026–5033.

[5] M. M. et. al., "Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning," *arXiv preprint arXiv:2511.04831*, 2025. [Online]. Available: https://arxiv.org/abs/2511.04831

[6] C. J. K. L. S. O. Jorge O, Arjun V, "Inhand manipulation allegro," https://github.com/arjun-2612/AssistiveRobotics_FP, 2025, accessed: December 12, 2025.

[7] ——, "Inhand manipulation allegro presentation," https://docs.google.com/presentation/d/10T0Uc71eYab5WoZrxiLx-OfGIqihncT6855RK-uiIPs/edit?usp=sharing, 2025, accessed: December 12, 2025.

Term	Expression
Steps Per Episode	24
Max Iterations	5000
Save Interval [iter]	50
Empirical Normalization	False
Policy Noise Std	1.0
Actor Network Hidden Dim	$\begin{bmatrix}512 & 256 & 128\end{bmatrix}$
Critic Network Hidden Dim	$\begin{bmatrix}512 & 256 & 128\end{bmatrix}$
Learning Rate	1e-3
Network Activation	Elu
Value Loss Coefficient	1.0
Clipped Value Loss	True
Clip Parameter	0.2
Entropy Coefficient	0.2
Gamma	0.99
Lambda	0.95
Desired KL	0.01
Max Gradient Norm	1.0

Table 1: PPO Experiment Hyperparamters

Term	Expression
Joint positions to limit normalized (rads)	$\frac{2*(q-(0.5(l+u)))}{u-l}$
Joint velocity (rads/s)	$\dot{q}$
Object position [x,y,z] (m)	p_o^W
Object quaternion [w,x,y,z]	$\hat{q}_o^W$
Object linear velocity [x,y,z] (m/s)	v_o^W
Object angular velocity [x,y,z] (rads/s)	ω_o^W
Goal pose [w,x,y,z]	q_o^W
Goal quaternion difference [w,x,y,z]	$\hat{q}_o^W * \bar{q}_o^W$
Previous action [rad/s]	a_{t-1}

Table 2: Observation Vector

Term	Expression	Weight
Orientation tracking	$\|\|AngleAxis(\hat{q}_o^W * \bar{q}_o^W)\|\|$	1.0
Success bonus	$\|\|AngleAxis(\hat{q}_o^W * \bar{q}_o^W)\|\| <= d_{thresh}$	250.0
Joint velocity	$\|\|\dot{\mathbf{q}}\|\|$	$-2.5e^{-5}$
Action magnitude	$\|\|a_t\|\|$	-0.0001
Action rate	$\|\|a_t - a_{t-1}\|\|$	-0.01

Table 3: Rewards and Penalties

Term	Expression
Timeout	$t > T_{episode}$
Max consecutive successes	$n_{success} > 50$
Object out of reach	$\|\|p_o^W - p_r^W\|\| > 0.03$

Table 4: Terminations

Term	Range
Absolute Robot Body Friction	$(0.7, 1.3)$
Scaled Robot Base Mass [kg]	$(0.95, 1.05)$
Scaled Robot Joint Stiffness	$(0.3, 3.0)$
Scaled Robot Joint Damping	$(0.75, 1.5)$
Absolute Object Body Friction	$(0.7, 1.3)$
Scaled Object Mass [kg]	$(0.4, 1.6)$
Reset Object Position XYZ [m]	$(-0.01, 0.01)$

Table 5: Domain Randomization Table

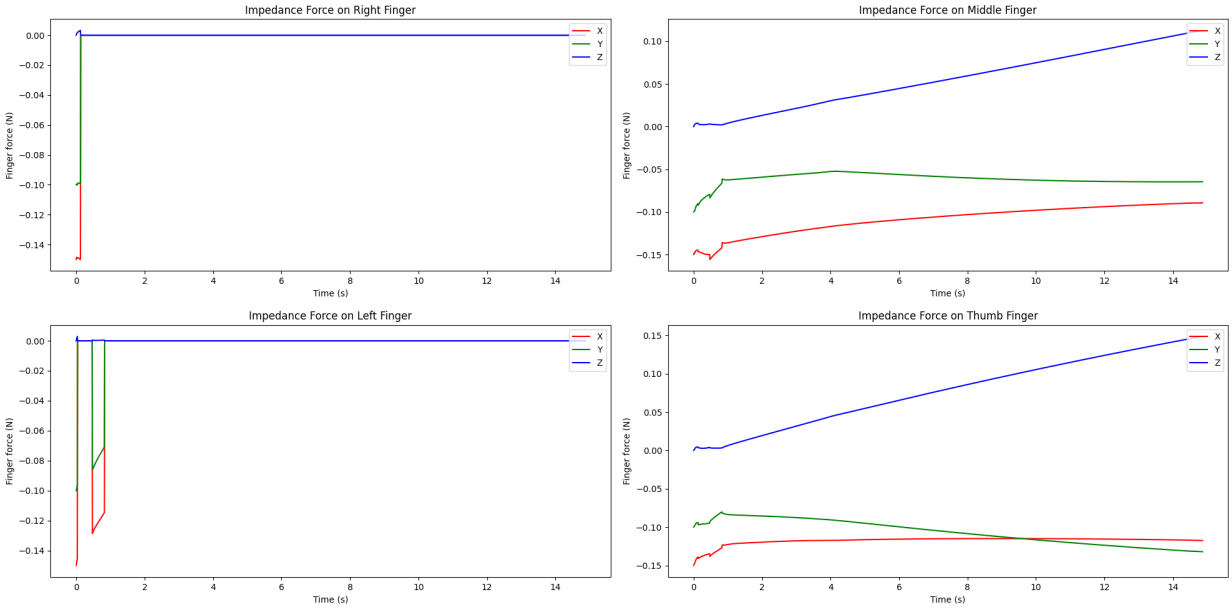


Figure 8: Impedance Controller Forces on Each Finger

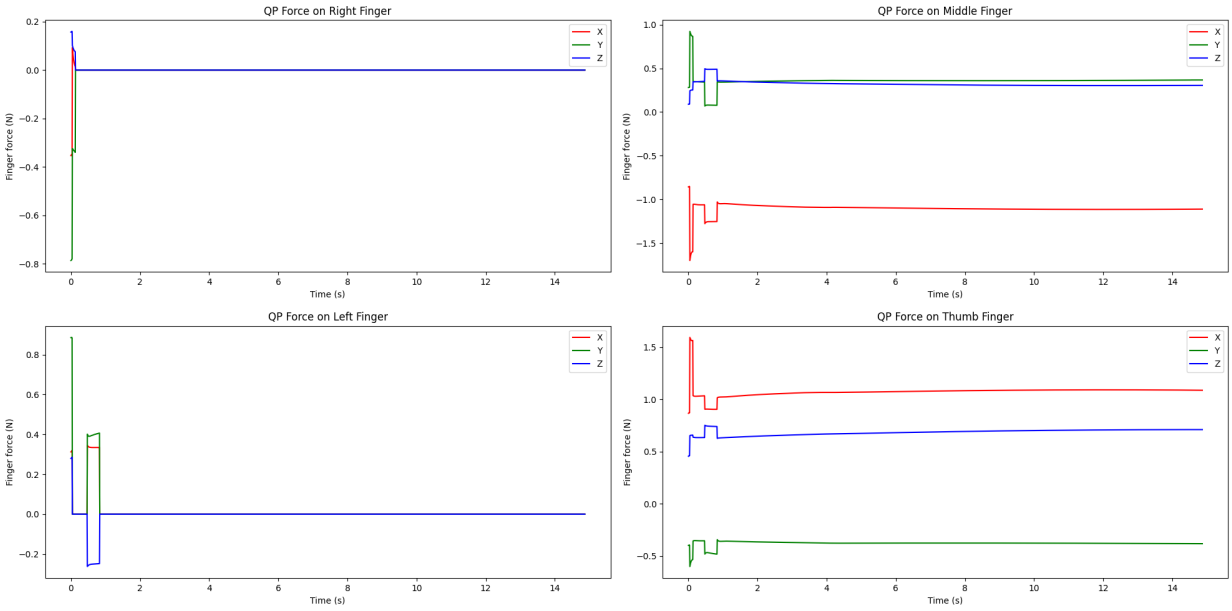


Figure 9: QP Forces on Each Finger

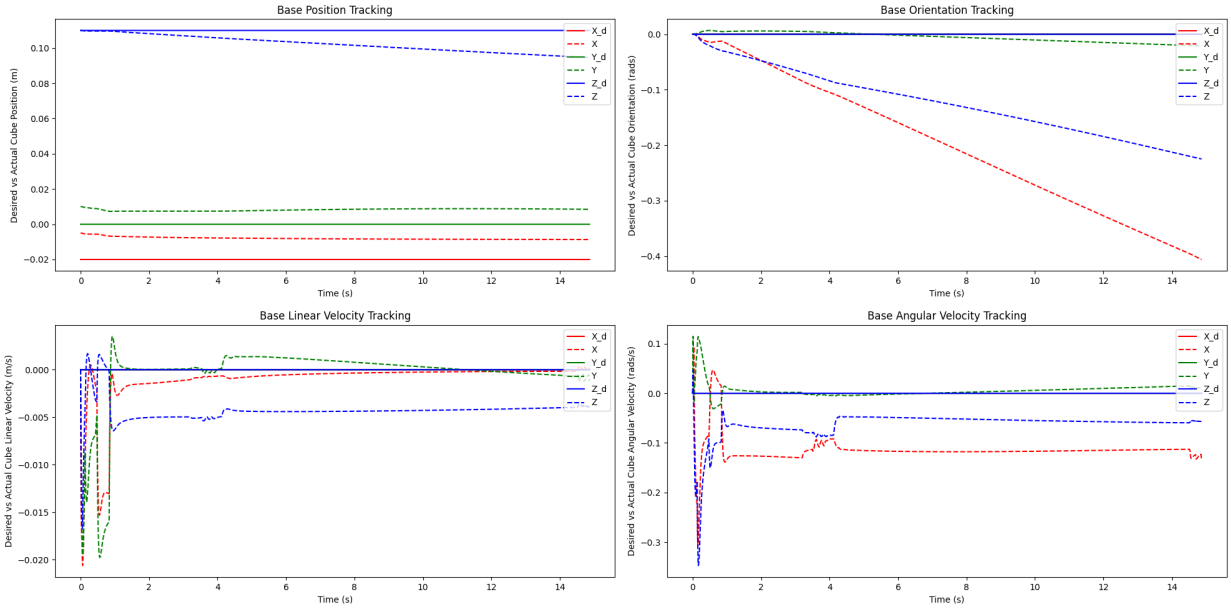


Figure 10: Object State Tracking with Force Control

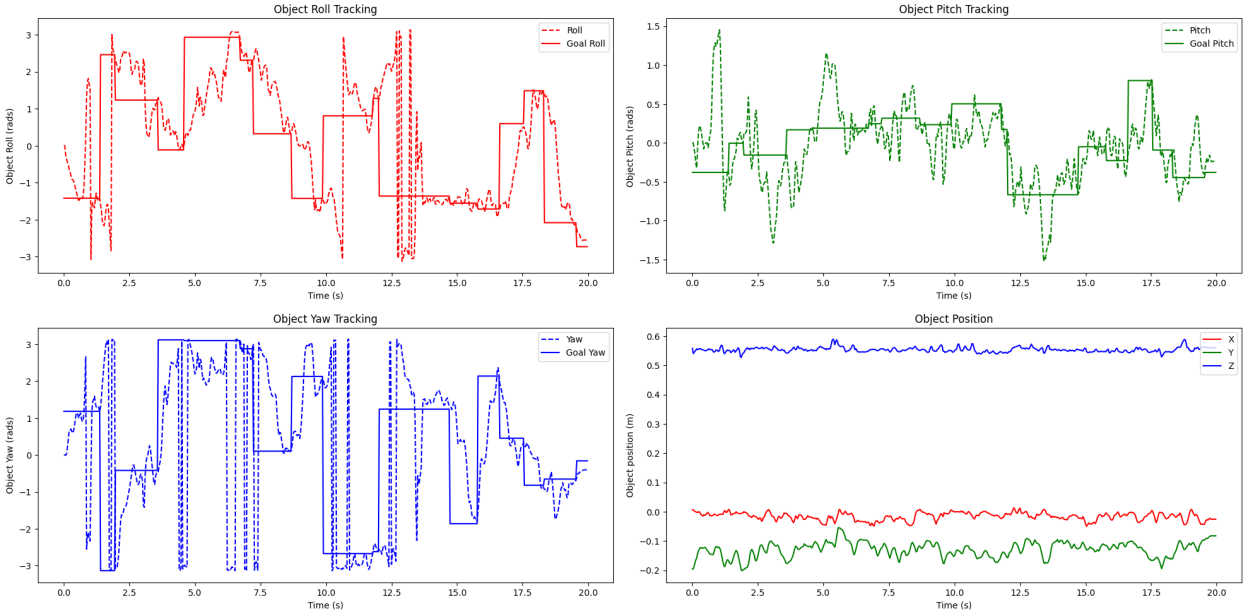


Figure 11: Object Pose Tracking with RL Policy